Databases
Storage Buckets
Monitoring
Templates
Create
Best Practices
Publish and Share
Deploy
Public API
Manage Projects
Manage Services
Manage Deployments
Manage Variables
Config as Code
Node.js SIGTERM

**Tutorials**

Getting Started
Technical Tutorials
GitHub Actions
GitLab

**Reference**

Errors
Develop
Deploy
Monitoring
Operations
CLI API
MCP Server
Public API
Templates
Pricing
Migrate to Railway Metal
Production Readiness Checklist

**Maturity**

Philosophy
Use Cases
Compliance
Enterprise
Incident Management
Compare to Heroku
Compare to Render
Compare to Fly
Compare to Vercel
Compare to DigitalOcean
Compare to VPS

# Template Best Practices

Copy as Markdown

Creating templates can get complex, but these best practices will help you create templates that are easy to use and maintain.

## Checklist

Depending on the type of template, there are different things to consider:

- Template and Service Icons
- Naming Conventions
- Private Networking
- Environment Variables
- Health Checks
- Persistent Storage
- Authentication
- Dry Code
- Workspace Naming
- Overview

## Template and Service Icons

Template and service icons are important for branding and recognition, as they give the template a more professional look and feel.

Always use 1:1 aspect ratio icons or logos with transparent backgrounds for both the template itself and the services the template includes.

Transparent backgrounds ensure logos integrate seamlessly with Railway's interface and provide a more polished, professional appearance.

## Naming Conventions

Naming conventions are important for readability and consistency; using proper names enhances the overall quality and credibility of your template.

Always follow the naming conventions for the software that the template is made for.

Example, if the template is for ClickHouse, the service and template name should be named `ClickHouse` with a capital C and H, since that is how the brand name is spelled.

For anything else, such as custom software:

- Use capital case.
- Avoid using special characters or dashes, space-delimited is the way to go.
- Prefer shorter names over longer names for better readability.
- Keep names concise while maintaining clarity.

## Private Networking

Private networking provides faster, free communication between services and reduces costs compared to routing traffic through the public internet.

On This Page

Checklist
Template and Service Icons
Naming Conventions
Private Networking
Environment Variables
Health Checks
Persistent Storage
Authentication
Dry Code
Workspace Naming
Overview

Always configure service-to-service communication (such as backend to database connections) to use private network hostnames rather than public domains.

For more details, see the private networking guide and reference documentation.

## Environment Variables

Properly set up environment variables are a great way to increase the usability of your template.

When using environment variables:

- Always include a description of what the variable is for.
- If a variable is optional, include a description explaining its purpose and what to set it to or where to find its value.
- For any secrets, passwords, keys, etc., use template variable functions to generate them, avoid hardcoding default credentials at all costs.
- Use reference variables when possible for dynamic service configuration.
- When referencing a hostname, use the private network hostname rather than the public domain, e.g., `RAILWAY_PRIVATE_DOMAIN` rather than `RAILWAY_PUBLIC_DOMAIN`.
- Include helpful pre-built variables that the user may need, such as database connection strings, API keys, hostnames, ports, and so on.

## Health Checks

Health checks are important for ensuring that the service is running properly, before traffic is routed to it.

Although a health check might not be needed for all software, such as Discord bots, when it is applicable, it is a good idea to include a health check.

A readiness endpoint is the best option; if that's not possible, then a liveness endpoint should be used.

For more details, see the healthchecks guide and reference documentation.

## Persistent Storage

Persistent storage is essential for templates that include databases, file servers, or other stateful services that need to retain data across deployments.

Always include a volume for these services.

Without persistent storage, data will be lost between redeployments, leading to unrecoverable data loss for template users.

When configuring the service, ensure the volume is mounted to the correct path. An incorrect mount path will prevent data from being stored in the volume.

Some examples of software that require persistent storage:

- **Databases:** PostgreSQL, MySQL, MongoDB, etc.
- **File servers:** Nextcloud, ownCloud, etc.
- **Other services:** Redis, RabbitMQ, etc.

The volume mount location depends entirely on where the software expects it to be mounted. Refer to the software's documentation for the correct mount path.

For more details, see the volumes guide and reference documentation.

## Authentication

Authentication is a common feature for many software applications, and it is crucial to properly configure it to prevent unauthorized access.

If the software provides a way to configure authentication, such as a username and password, or an API key, you should always configure it in the template.

For example, ClickHouse can operate without authentication, but authentication should always be configured so as not to leave the database open and unauthenticated to the public.

Passwords, API keys, etc. should be generated using template variable functions, and not hardcoded.

## Dry Code

This is most applicable to templates that deploy from GitHub.

When creating templates that deploy from GitHub, include only the essential files needed for deployment. Avoid unnecessary documentation, example files, or unused code and configurations that don't contribute to the core functionality.

A clean, minimal repository helps users quickly understand the template's structure and make customizations when needed.

## Workspace Naming

When users deploy a template, the template author appears as the name of the workspace that created and published it.

Since the author is publicly visible and shown with the template to the users, it is important to make sure the workspace name is professional and **accurately represents your relationship to the software**.

**Important:** Only use a company or organization name as your workspace name if you are officially authorized to represent that entity. Misrepresenting your affiliation is misleading to users and violates trust.

**If you are officially affiliated** with the software (e.g., you work for ClickHouse and are creating a ClickHouse template), then using the official company name "ClickHouse, Inc." is appropriate and helpful for users to identify official templates.

**If you are not officially affiliated** with the software, always use your own professional name as the workspace name.

**Note:** To transfer a template from one workspace to another, contact support.

## Overview

The overview is the first thing users will see when they click on the template, so it is important to make it count.

The overview should include the following:

- **H1: Deploy and Host [X] with Railway**

  What is X? Your description in roughly ~ 50 words.

- **H2: About Hosting [X]**

Roughly 100 word description what's involved in hosting/deploying X

- **H2: Common Use Cases**

  In 3-5 bullets, what are the most common use cases for [X]?

- **H2: Dependencies for [X] Hosting**

  In bullet form, what other technologies are incorporated in using this template besides [X]?

- **H3: Deployment Dependencies**

  Include any external links relevant to the template.

- **H3: Implementation Details (Optional)**

  Include any code snippets or implementation details. This section is optional. Exclude if nothing to add.

- **H3: Why Deploy [X] on Railway?**

  Railway is a singular platform to deploy your infrastructure stack. Railway will host your infrastructure so you don't have to deal with configuration, while allowing you to vertically and horizontally scale it.

  By deploying [X] on Railway, you are one step closer to supporting a complete full-stack application with minimal burden. Host your servers, databases, AI agents, and more on Railway.